

Job Agent

求职与职业转型助手 Agent

从 JD 和简历输入，到匹配分析、证据定位、简历改写和面试准备的本地 AI Agent workflow。

Python

Streamlit

PDF 解析

OCR

评测集

Mock LLM

项目核心定位

这不是一个单次 prompt demo，而是一个完整 AI 产品闭环：输入处理、Agent 编排、规则匹配、结构化输出、Web UI、评测回归和真实数据迭代。

0-1

完整构建

从空项目到本地可运行 AI Agent

95/100

真实样例

中文 JD 与 PDF 简历匹配结果

41

自动化测试

覆盖核心流程与 Web UI 辅助逻辑

3/3

评测通过

高 / 中 / 低匹配样例全部通过

项目摘要

用真实求职问题，搭建可解释、可演示、可回归的 AI Agent。

项目背景

求职者在投递岗位前，通常很难判断自己的简历和目标 JD 到底匹不匹配，也不知道应该优先补强哪段经历。Job Agent 从这个真实痛点出发，把“凭感觉判断”转成“逐条要求 + 简历证据 + 可行动建议”。

6 类输入

TXT / MD / RTF / PDF / 图片 / macOS 文本包

中文 OCR

默认 chi_sim+eng，支持 JD 截图识别

结构化输出

Markdown 给人看，JSON 给系统接

零成本演示

默认 mock 模式，无需 API Key

这个项目证明了什么

- 能把一个真实用户问题拆成完整 AI 产品流程，而不是停留在概念层。
- 能处理真实文件输入，包括 PDF 简历、截图 OCR 和异常文本格式。
- 能让结果可解释，每条岗位要求都能看到对应简历证据。
- 能用评测和测试守住迭代质量，避免只对单一样例有效。

问题定义

把模糊的求职判断，变成可解释的匹配证据。

用户痛点

- JD 很长，不知道真正关键的要求是什么。
- 简历内容很多，不知道哪段经历最能支撑目标岗位。
- 只能凭感觉判断匹配度，缺少逐条证据。
- 即使知道有差距，也不知道简历 bullet 应该怎么改。

产品目标

- 读懂岗位：从 JD 中提取关键要求。
- 找到证据：从简历中定位能支撑要求的经历。
- 判断差距：输出匹配分、已匹配项和待补强项。
- 辅助改写：生成可参考的中文简历 bullet。
- 准备面试：围绕岗位要求生成准备问题。

设计原则

Job Agent 不替用户编造经历，而是把已有简历和目标岗位进行结构化对齐。它强调“可解释”和“可行动”：用户能看到为什么匹配、哪里不够、下一步简历该怎么表达。

Agent workflow

不是一次 prompt，而是多步骤任务编排。

01

输入 JD 和简历

支持文本、PDF、截图和 macOS 文本包。

02

读取与清洗

处理文件校验、PDF 文本、OCR 空格和联系方式过滤。

03

解析岗位要求

提取岗位标题、职责和任职要求。

04

提取简历亮点

识别更可能支撑岗位要求的经历句。

05

逐条匹配证据

把 JD 要求和简历证据关联，并计算匹配强度。

06

生成报告

输出匹配分、缺口、证据解释和改写建议。

07

导出 JSON

保留结构化结果，便于后续系统集成和评测。

08

Web UI 展示

让非技术用户可以上传文件并查看结果。

核心能力拆解

围绕真实求职数据，补齐输入、判断和输出能力。

01

多格式输入

支持 TXT、MD、RTF、PDF、图片截图和 macOS 文本包，降低真实使用门槛。

02

中文 OCR 与清洗

默认 chi_sim+eng，处理中文截图识别后的异常空格，提升 JD 解析稳定性。

03

可解释匹配

不只输出分数，还逐条说明 JD 要求对应到简历里的哪一句证据。

04

简历 bullet 改写

区分当前依据、建议写法和改写理由，把分析结果变成可行动建议。

05

本地 Web UI

支持样例、真实样例和上传文件分析，面试展示更直观。

06

评测回归

高 / 中 / 低匹配样例和 41 个 unittest 支撑持续迭代。

真实数据迭代

从真实数据问题中迭代，而不是只跑理想样例。

问题	暴露现象	解决动作
macOS 文本包	看起来是 txt，实际是包含 TXT.rtf 的目录。	增加文本包读取能力。
PDF 断行	简历经历被异常换行切断，证据句不完整。	增加基础中文断行合并。
中文匹配弱	初始规则偏英文，真实样例只有 18/100。	补中文 JD 章节识别和能力词组。
报告混杂	中英文混排，阅读成本高。	改成中文口语化报告。
上传方式真实化	用户更可能上传 JD 截图和 PDF 简历。	补 OCR、PDF 和 Web 上传入口。

迭代结果

18/100 → 95/100

真实中文岗位样例：智能化产品经理（G端）。当前已匹配 6 条岗位要求，待补强要求 0 条，并生成 3 条可参考简历 bullet。

评测与质量保障

用评测集和测试，保证 Agent 迭代可回归。

评测集结果

high_match

82

Expected 80-100

PASS

medium_match

55

Expected 45-79

PASS

low_match

9

Expected 0-44

PASS

41 个 unittest 覆盖

- 输入读取、PDF / OCR 辅助逻辑、JD 解析和简历解析。
- 匹配引擎、Agent 主流程、报告与 JSON 输出。
- Web UI 辅助逻辑、上传入口文案、复制按钮和评测流程。

我想表达的是：AI 项目不能只看一次生成效果，还需要有评测样例、回归测试和稳定演示路径。

Demo 展示与产物

从命令行到 Web UI，形成可演示的本地产品原型。

Web UI 三种输入方式

- 使用内置样例：证明基础流程稳定。
- 使用真实样例：展示中文 JD + PDF 简历的真实效果。
- 上传自己的文件：支持 JD 文件 / 截图和简历文件。

Markdown 报告

给人阅读，适合演示和作品集展示。

JSON 结果

给程序读取，适合后续接数据库、Web 页面和批量分析。

评测 summary

证明项目具备回归验证能力。

建议截图清单

- Web UI 首页和价值卡片。
- 真实样例分析结果：匹配分和结论。
- 逐条证据页签：JD 要求和简历证据。
- 简历改写页签：建议写法和复制 icon。
- 评测 summary：3 组样例全部通过。

我的角色与能力沉淀

这个项目沉淀的是 AI 产品从 0 到 1 的完整构建能力。

我在项目中的角色

- 定义求职匹配的用户问题和产品边界。
- 拆解 Agent 工作流，而不是只写单次 prompt。
- 设计输入、解析、匹配、输出、评测和 Web UI 的模块边界。
- 用真实 JD 和简历暴露问题，并迭代中文匹配、PDF 清洗和报告表达。
- 保持默认 mock 模式，控制成本和演示稳定性。
- 通过测试、评测集和文档记录，让项目可以被复现、解释和继续扩展。

能力关键词

AI 产品定义

Agent 工作流设计

真实数据迭代

结构化输出

评测回归

本地 Demo 交付

成本控制

适合简历的一句话

独立设计并实现本地求职场景 AI Agent，支持 JD/简历多格式输入、匹配分析、证据定位、简历 bullet 改写、Web UI 展示和评测回归。

局限与下一步

敢于讲清楚边界，项目会比“什么都能做”更可信。

当前限制

- 当前匹配机制是规则增强版，不是深度语义模型。
- 复杂 PDF、多栏排版和扫描件仍需要更强清洗策略。
- 真实截图 OCR 本轮暂不作为交付阻塞项。
- 真实 LLM 接入暂停，默认 mock 模式保证稳定演示。
- 当前是本地 Web UI，还没有公网部署和历史记录。

下一步规划

- 短期：整理作品集截图和演示脚本，初始化 git 仓库。
- 中期：接入真实 LLM，优化 bullet 润色和语义判断。
- 长期：支持多岗位批量比较、投递优先级和面试提纲。

30 秒面试介绍

我做了一个本地运行的求职场景 AI Agent。它可以读取岗位 JD 和简历 PDF，自动提取岗位要求、定位简历证据、计算匹配度，并生成中文简历 bullet 改写和面试准备问题。这个项目不是单次 prompt，而是完整覆盖输入处理、Agent 编排、匹配逻辑、结构化输出、Web UI 和评测回归。